RETAILMART DATA PIPELINE

End-to-End Medallion Architecture Documentation

Organization: Celebal Technologies

Role: Data Engineering Intern

Platform: Databricks (Unity Catalog) + Pre-processing Layer

Type: Batch-Oriented, Incremental Hybrid Pipeline

Table of Contents

1. Project Overview & My Objective

This technical report documents the end-to-end production data pipeline developed during my Data Engineering internship at Celebal Technologies. The primary objective was to architect and implement a functional, scalable data warehouse framework directly on a cloud infrastructure.Using the industry-standard **Medallion Architecture (Bronze to Silver to Gold)**, the pipeline ingests raw, unstructured operational e-commerce transactional logs, applies structural data cleansing strategies, and transforms them into an optimized Star Schema model. This ensures that business analysts and managers can seamlessly execute high-performance analytical queries to extract critical operational insights.

1.1 Data Source Characterization

The pipeline processes three foundational e-commerce entity logging files:

- **Customers Data (customers.csv):** Contains demographic records and account identification attributes.
- **Products Data (products.csv):** Stores the retail merchandise catalog, including product categories and price indexes.
- **Orders Data (orders.csv):** Represents transactional log entries tracking sequential operational records, order dates, and purchase quantities.

2. Layer-by-Layer Implementation

2.1 Pre-processing & Cleansing Layer (Pandas Engine)

The initial operational e-commerce logs are pulled into the local environment via workspace system pathing. Utilizing a Pandas engine, messy datasets are parsed and standardized. A critical business metric, **delivery_days**, is computed by executing a temporal delta calculation (delivery_date - order_date). Rows containing invalid key constraints or full redundancies are stripped out, and the clean intermediate data assets are stored within the data/cleaned/ path.

2.2 Bronze Layer (Data Ingestion & Structural Locking)

In the first phase of the Medallion pipeline, the pre-processed CSV assets are ingested into Databricks using the Delta Lake format. To safeguard production jobs against sudden

structure failures, a deliberate defensive design strategy is applied: **all columns are strictly cast into a uniform STRING data type**. This schema enforcement setup neutralizes formatting variations. Concurrently, a technical audit column, lakehouse_ingestion_at, logs real-time ingestion timestamps.

2.3 Silver Layer (Validation, Type Casting & History Tracking)

The Silver Layer shifts the focus toward refining quality, applying type transformations, and restructuring data arrays:

- **Type Restoration:** Uniform string blocks are cast back into their correct native structures (e.g., retail pricing to double, sales quantity and delivery deltas to int).
- **SCD Type 2 History Tracking:** To preserve temporal variations in dimensions, structural metadata tracking attributes are added (is_current, effective_start_date, effective_end_date).
- **Surrogate Key Generation:** For optimal lookup performance and data anonymity, sequence-based immutable Surrogate Keys (customer_sk, product_sk, order_sk) are automatically generated using PySpark Window functions and the row_number() method.

2.4 Gold Layer (Star Schema Presentation Warehouse)

The Gold Layer serves as the final reporting and BI data tier. Cleaned Silver tables are decoupled and systematically structured into a **Star Schema Model**:

- **Dimension Tables:** dim_customer and dim_product tables are isolated to store full historical context and user profiles.
- **Fact Table:** A centralized **fact_orders** hub is compiled. It maps directly to the unique keys (customer_sk, product_sk) and calculates operational revenue KPIs, specifically **total_sales_amount** (computed as quantity * price).

3. Data Cleaning & Quality Checks

During the project, I implemented several checks to ensure the data was accurate:

Issue	Operational Fix Strategy (PySpark/Pandas)
Missing Primary Tracking Keys	Isolated and removed zero-identity records (e.g., missing customer IDs) using custom dropna() validation filters.
Operational Record Redundancy	Applied structural deduplication via dropDuplicates() across core tracking keys to resolve duplicate logging overlaps.
Schema Drift & Data Type Crashes	Implemented a strict STRING type casting rule at the Bronze layer to prevent automated runtime truncation errors.
Incomplete Business Metric Gaps	Standardized incomplete delivery loops by replacing unresolved null strings with a designated default flag indicator (-1).

4. Tools & Technology Stack I Learned

- **Databricks Ecosystem:** Utilized Python, Pandas, and PySpark DataFrame APIs to orchestrate data transformation workloads on scalable compute clusters.
- **Delta Lake Storage Engine:** Leveraged the Delta format to maintain ACID transactional guarantees and perform schema enforcement during parallel data writes.
- **Unity Catalog Governance:** Structured a multi-tier namespace schema environment (retailmart_catalog.operational_lakehouse) to ensure proper access management and secure tracking of storage entities.
- **Databricks SQL Warehouse:** Deployed optimized SQL compute clusters to handle high-performance procedural queries and complex data aggregation tasks.

5. Business Insights & Analytics KPIs (SQL Engine)

Once the Gold star schema is populated, the Databricks SQL engine runs advanced analytic scripts to extract five crucial business parameters:

1. **Order Delivery Volume Optimization:** Employs WHERE filters to isolate successful shipments, allowing logistics managers to evaluate warehouse efficiency.

2. **Customer 360 Revenue Yield:** Uses complex JOIN and GROUP BY logic to aggregate buyer metrics, uncovering individual lifetime order frequencies and overall financial contributions.
3. **Behavioral Customer Loyalty Segments:** Implements conditional CASE WHEN blocks to partition customers into distinct behavioral categories based on purchase velocity (*Elite Loyalty Tier*, *Regular Core Tier*, *Standard Entry Tier*).
4. **High-Value Transactions Isolation:** Executes complex subqueries to flag bulk business orders that exceed the platform's historical average basket size.
5. **Product Merchandise Performance Ranks:** Combines Common Table Expressions (CTEs) and analytical partition models (DENSE_RANK() OVER (PARTITION BY category ORDER BY ... DESC)) to pinpoint and rank the top-selling items within every separate market segment.

6. Notebook Execution Sequence

To automate dependency orchestration, the notebooks must be executed in this exact sequential order:

1. **1_data_generation.ipynb** (Generates mock datasets and provisions raw file environment logging)
2. **2_pandas_cleaning.ipynb** (Handles parsing, data scrubbing, and business metric generation)
3. **3_medallion_pipeline.ipynb** (Executes Bronze ingestion, Silver SK mappings, and Gold Star Schema compilation)
4. **4_business_insights.sql** (Runs pure SQL queries to generate operational reports and tables)

7. End-to-End Data Flow

CSV Log Files (Local Workspace Storage)

Pandas Pre-processing & Delivery Metric Formulation (data/cleaned/)

⬇

Bronze Layer Delta Tables (String Format Protection & Ingestion Timestamp)

⬇

Silver Layer Delta Tables (Data Type Restoration, SCD2 Tracking & Surrogate Keys)

⬇

Gold Layer Star Schema (Decoupled Dimensions & Consolidated Fact Table)

⬇

Databricks SQL Analytical Inquiries (Business Dashboard Report Generation)

8. Conclusion & Key Learnings

Building the end-to-end RetailMart analytical data pipeline has provided an outstanding, hands-on data warehousing experience. This capstone project allowed me to bridge the gap between classroom theory and enterprise-level real-world implementations.

My most significant learning curves involved mastering **Medallion Architecture processing rules, designing robust automated schema locks, and managing immutable sequence-based Surrogate Keys**. I am deeply grateful to my technical mentors at Celebal Technologies for their invaluable support, guidance, and technical review throughout this internship project!